H&M Fashion POS - GitHub Project Code Documentation

File Name: app.py

```
import streamlit as st
import pandas as pd
import database as db

st.set_page_config(page_title="H&M Fashion POS", layout="wide", initial_sidebar_state="expanded")

db.init_db()

st.sidebar.title("H&M FASHION POS")
menu = st.sidebar.radio("Navigation", ["Checkout", "Inventory", "Sales Report"])

if menu == "Checkout":
    st.header("POS Checkout")

    if "cart" not in st.session_state:
        st.session_state.cart = []

    col1, col2 = st.columns([2, 1])

    with col1:
        search = st.text_input("Search Product by Name or Barcode", key="pos_search")
        products = db.get_products(search)

        st.subheader("Products")
        if not products.empty:
            for idx, row in products.iterrows():
                p_col1, p_col2, p_col3, p_col4 = st.columns([3, 2, 2, 2])
                p_col1.write(f"**{row['name']}** ({row['size']} / {row['color']})")
                p_col2.write(f"Barcode: {row['barcode']}")
                p_col3.write(f"${row['price']:.2f} | Stock: {row['stock_quantity']}")
                if p_col4.button("Add to Cart", key=f"add_{row['product_id']}"):
                    if row['stock_quantity'] > 0:
                        found = False
                        for item in st.session_state.cart:
                            if item["id"] == row['product_id']:
                                item["qty"] += 1
                                item["subtotal"] = item["qty"] * item["price"]
                                found = True
                                break
                        if not found:
                            st.session_state.cart.append({
                                "id": row['product_id'],
                                "name": row['name'],
                                "price": row['price'],
                                "qty": 1,
                                "subtotal": row['price']
                            })
                        st.rerun()
                    else:
                        st.error("Out of stock!")
        else:
            st.info("No products found.")

    with col2:
```

```
        st.subheader("Current Order")
        if st.session_state.cart:
            cart_df = pd.DataFrame(st.session_state.cart)
            st.dataframe(cart_df[["name", "qty", "price", "subtotal"]], use_container_width=True)

            gross_total = sum(item["subtotal"] for item in st.session_state.cart)
            st.write(f"### Gross Total: ${gross_total:.2f}")

            discount = st.number_input("Discount ($)", min_value=0.0, value=0.0, step=0.5)
            final_total = max(0.0, gross_total - discount)
            st.write(f"## Final Total: ${final_total:.2f}")

            pay_method = st.selectbox("Payment Method", ["Cash", "Card", "KBZPay", "CBPay", "AYA Pay"])

            if st.button("Clear Cart"):
                st.session_state.cart = []
                st.rerun()

            if st.button("Complete Sale", type="primary"):
                      sale_id = db.process_checkout(st.session_state.cart, gross_total, discount, final_total,
pay_method)
                st.success(f"Sale #{sale_id} completed successfully!")
                st.session_state.cart = []
                st.rerun()
        else:
            st.info("Cart is empty.")

elif menu == "Inventory":
    st.header("Inventory Management")

    with st.expander("Add New Product", expanded=False):
        with st.form("add_product_form"):
            c1, c2 = st.columns(2)
            barcode = c1.text_input("Barcode")
            name = c2.text_input("Product Name")
            category = c1.text_input("Category")
            size = c2.text_input("Size (e.g. S, M, L, XL)")
            color = c1.text_input("Color")
            price = c2.number_input("Price ($)", min_value=0.0, step=0.01)
            stock = c1.number_input("Stock Quantity", min_value=0, step=1)

            submit = st.form_submit_button("Save Product")
            if submit:
                if name and price >= 0:
                    try:
                        db.add_product(barcode, name, category, size, color, price, stock)
                        st.success(f"Product '{name}' added successfully!")
                        st.rerun()
                    except Exception as e:
                        st.error(f"Error adding product: {e}")
                else:
                    st.warning("Please fill in required fields (Name, Price).")

    st.subheader("All Products")
    products = db.get_products()
    st.dataframe(products, use_container_width=True)
```

```
elif menu == "Sales Report":
    st.header("Sales Report & History")
    sales_df = db.get_sales_report()

    if not sales_df.empty:
        total_revenue = sales_df["final_amount"].sum()
        total_sales_count = len(sales_df)

        m1, m2 = st.columns(2)
        m1.metric("Total Revenue", f"${total_revenue:.2f}")
        m2.metric("Total Transactions", total_sales_count)

        st.subheader("Transaction History")
        st.dataframe(sales_df, use_container_width=True)
    else:
        st.info("No sales records found yet.")
```

H&M Fashion POS - GitHub Project Code Documentation

File Name: database.py

```
import sqlite3
import pandas as pd
from datetime import datetime

DB_NAME = "hm_fashion_pos.db"

def get_connection():
    return sqlite3.connect(DB_NAME)

def init_db():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.executescript('''
    CREATE TABLE IF NOT EXISTS products (
        product_id INTEGER PRIMARY KEY AUTOINCREMENT,
        barcode TEXT UNIQUE,
        name TEXT NOT NULL,
        category TEXT,
        size TEXT,
        color TEXT,
        price REAL NOT NULL,
        stock_quantity INTEGER NOT NULL DEFAULT 0
    );

    CREATE TABLE IF NOT EXISTS sales (
        sale_id INTEGER PRIMARY KEY AUTOINCREMENT,
        sale_date TEXT NOT NULL,
        total_amount REAL NOT NULL,
        discount REAL DEFAULT 0,
        final_amount REAL NOT NULL,
        payment_method TEXT DEFAULT 'Cash'
    );

    CREATE TABLE IF NOT EXISTS sale_items (
        item_id INTEGER PRIMARY KEY AUTOINCREMENT,
        sale_id INTEGER,
        product_id INTEGER,
        quantity INTEGER NOT NULL,
        unit_price REAL NOT NULL,
        subtotal REAL NOT NULL
    );
    ''')
    conn.commit()
    conn.close()

def get_products(search_term=""):
    conn = get_connection()
    if search_term:
        query = "SELECT * FROM products WHERE barcode LIKE ? OR name LIKE ? OR category LIKE ?"
                    df   =   pd.read_sql_query(query,   conn,   params=(f"%{search_term}%",   f"%{search_term}%",
f"%{search_term}%"))
    else:
        df = pd.read_sql_query("SELECT * FROM products", conn)
    conn.close()
```

```
    return df

def add_product(barcode, name, category, size, color, price, stock):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO products (barcode, name, category, size, color, price, stock_quantity)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (barcode, name, category, size, color, price, stock))
    conn.commit()
    conn.close()

def process_checkout(cart, gross_total, discount, final_total, pay_method):
    conn = get_connection()
    cursor = conn.cursor()
    sale_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    cursor.execute('''
        INSERT INTO sales (sale_date, total_amount, discount, final_amount, payment_method)
        VALUES (?, ?, ?, ?, ?)
    ''', (sale_date, gross_total, discount, final_total, pay_method))
    sale_id = cursor.lastrowid

    for item in cart:
        cursor.execute('''
            INSERT INTO sale_items (sale_id, product_id, quantity, unit_price, subtotal)
            VALUES (?, ?, ?, ?, ?)
        ''', (sale_id, item["id"], item["qty"], item["price"], item["subtotal"]))
        cursor.execute('''
            UPDATE products SET stock_quantity = stock_quantity - ? WHERE product_id = ?
        ''', (item["qty"], item["id"]))

    conn.commit()
    conn.close()
    return sale_id

def get_sales_report():
    conn = get_connection()
    df = pd.read_sql_query("SELECT * FROM sales ORDER BY sale_id DESC", conn)
    conn.close()
    return df
```

H&M Fashion POS - GitHub Project Code Documentation

File Name: requirements.txt

```
streamlit
pandas
```